

```
"""
```

```
|| ALPHA SIGNALS BOT - Low Cap Hunter ||  
|| بوت إشارات ألفا - صياد العملات الصغيرة ||  
||
```

الإعدادات:

1. pip install python-telegram-bot requests aiohttp
2. CONFIG ضع التوكن واسم القناة في قسم
3. python alpha_signals_bot.py

```
"""
```

```
import asyncio  
import aiohttp  
import logging  
import time  
import json  
  
from datetime import datetime, timezone  
from telegram import Bot  
from telegram.constants import ParseMode  
  
# =====  
#  CONFIG - هذين السطرين فقط  
# =====  
  
BOT_TOKEN = "8735462840:AAF5uJI6w5ZVUjxqy58rpawLJP4X_9v51A8" # توكن البوت  
CHANNEL_ID = "@Sloomaw" # قناة التيليجرام  
  
# =====  
#  إعدادات التحليل (قابلة للتخصيص)  
# =====  
  
SCAN_INTERVAL_MINUTES = 15 # كل كم دقيقة يسكان  
MIN_VOLUME_USD = 50_000 # أدنى حد h حجم تداول 24  
MAX_MARKET_CAP_USD = 20_000_000 # ماركت كاب أقصى (Low Cap)  
MIN_MARKET_CAP_USD = 100_000 # فلتر ضد الروبيشات الصغيرة جداً  
MIN_VOLUME_MCAP_RATIO = 0.15 # زخم حقيقي - Volume/MCap  
MIN_PRICE_CHANGE_1H = 3.0 # ارتفاع في ساعة %  
MIN_PRICE_CHANGE_6H = 5.0 # ارتفاع في 6 ساعات %  
BREAKOUT_THRESHOLD = 8.0 # قوي Breakout ارتفاع يُعتبر %  
MIN_LIQUIDITY_USD = 30_000 # أدنى حد Uniswap/DEX سيولة  
BLACKLIST_KEYWORDS = [ # كلمات تُستبعد فوراً  
    "elon", "doge2", "moon", "safe", "inu",  
    "baby", "pepe2", "shib2", "cum", "porn",  
    "xxx", "fair", "gem", "100x", "1000x"
```

```

]
COOLDOWN_HOURS          = 4          # ساعات X لا تكرر نفس العملة قبل

# =====
# 📡 مصادر البيانات (مجانية بالكامل)
# =====

COINGECKO_BASE  = "https://api.coingecko.com/api/v3"
DEXSCREENER_BASE= "https://api.dexscreener.com/latest/dex"

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler("alpha_bot.log", encoding="utf-8"),
        logging.StreamHandler()
    ]
)

log = logging.getLogger("AlphaBot")

# =====
# 🧠 منطق التحليل
# =====

class SignalReason:
    """يبنى نص سبب الإشارة تلقائياً"""
    def __init__(self):
        self.points = []
        self.score = 0

    def add(self, emoji: str, text: str, pts: int = 1):
        self.points.append(f"{emoji} {text}")
        self.score += pts

    def build(self) -> str:
        return "\n".join(self.points)

def is_blacklisted(name: str, symbol: str) -> bool:
    combined = (name + symbol).lower()
    return any(kw in combined for kw in BLACKLIST_KEYWORDS)

def analyze_coingecko_coin(coin: dict) -> tuple[bool, SignalReason]:
    """CoinGecko تحليل عملة من"""
    r = SignalReason()

    mcap          = coin.get("market_cap") or 0

```

```

volume_24h = coin.get("total_volume") or 0
change_1h = coin.get("price_change_percentage_1h_in_currency") or 0
change_24h = coin.get("price_change_percentage_24h") or 0
change_7d = coin.get("price_change_percentage_7d_in_currency") or 0
name = coin.get("name", "")
symbol = coin.get("symbol", "").upper()

# === فلاتر أساسية ===
if is_blacklisted(name, symbol):
    return False, r
if not (MIN_MARKET_CAP_USD <= mcap <= MAX_MARKET_CAP_USD):
    return False, r
if volume_24h < MIN_VOLUME_USD:
    return False, r

vol_mcap_ratio = volume_24h / mcap if mcap else 0

# === شروط الدخول ===
passed = (
    change_1h >= MIN_PRICE_CHANGE_1H and
    change_24h >= MIN_PRICE_CHANGE_6H and
    vol_mcap_ratio >= MIN_VOLUME_MCAP_RATIO
)
if not passed:
    return False, r

# === بناء أسباب الإشارة ===
mcap_str = f"${mcap/1e6:.2f}M"
vol_str = f"${volume_24h/1e3:.0f}K"

r.add("📊", f"24 التداول الحجم: **{vol_str}** (نشاط عالي)", 2)
r.add("💎", f"فرصة تضاعف", 2, f"ماركت كاب صغير: **{mcap_str}** → 2")
r.add("⚡", f"زخم قوي", 2, f"ارتفاع 1h: **{change_1h:.1f}%** - 2")
r.add("📈", f"ارتفاع 24h: **{change_24h:.1f}%**", 1)

if vol_mcap_ratio > 0.5:
    r.add("🔥", f"ط شراء ضخم", 3, f"Volume/MCap: **{vol_mcap_ratio:.1%}** - 3")

if change_1h >= BREAKOUT_THRESHOLD:
    r.add("🚀", f"عنة!", 5, f"{BREAKOUT_THRESHOLD}% {BREAKOUT_THRESHOLD} اختراق قوي فوق - **BREAKOUT**")

if change_7d and change_7d < -10 and change_1h > 5:
    r.add("💡", f"Reversal انعكاس بعد تراجع أسبوعي - نمط", 3)

if change_24h > 15:
    r.add("🎯", f"(قوي جداً - مومنتم مستمر", 2, f"ارتفاع 24h")

```

```

return True, r

def analyze_dexscreener_pair(pair: dict) -> tuple[bool, SignalReason]:
    """تحليل زوج من DexScreener (DEX chains)"""
    r = SignalReason()

    try:
        liquidity = (pair.get("liquidity") or {}).get("usd") or 0
        volume_h24 = (pair.get("volume") or {}).get("h24") or 0
        fdv = pair.get("fdv") or 0
        change_h1 = (pair.get("priceChange") or {}).get("h1") or 0
        change_h6 = (pair.get("priceChange") or {}).get("h6") or 0
        change_h24 = (pair.get("priceChange") or {}).get("h24") or 0
        txns_h1 = (pair.get("txns") or {}).get("h1") or {}
        buys_h1 = txns_h1.get("buys", 0)
        sells_h1 = txns_h1.get("sells", 0)
        name = (pair.get("baseToken") or {}).get("name", "")
        symbol = (pair.get("baseToken") or {}).get("symbol", "").upper()
        dex = pair.get("dexId", "")
        chain = pair.get("chainId", "")
        pair_url = pair.get("url", "")
    except Exception:
        return False, r

    # == فلاتر أساسية ==
    if is_blacklisted(name, symbol):
        return False, r
    if liquidity < MIN_LIQUIDITY_USD:
        return False, r
    if volume_h24 < MIN_VOLUME_USD:
        return False, r
    if fdv > MAX_MARKET_CAP_USD or fdv < MIN_MARKET_CAP_USD:
        return False, r

    buy_pressure = buys_h1 / (buys_h1 + sells_h1) if (buys_h1 + sells_h1) > 0 else

    # == شروط الدخول ==
    passed = (
        change_h1 >= MIN_PRICE_CHANGE_1H and
        change_h6 >= MIN_PRICE_CHANGE_6H and
        buy_pressure >= 0.55
    )
    if not passed:
        return False, r

    # == بناء الأسباب ==

```

```

liq_str = f"${liquidity/1e3:.0f}K"
vol_str = f"${volume_h24/1e3:.0f}K"
fdv_str = f"${fdv/1e6:.2f}M"

r.add("🌊", f"سيولة DEX: **{liq_str}** - 2 , "سيولة كافية وآمنة", 2)
r.add("📊", f"حجم تداول 24h: **{vol_str}**", 2)
r.add("💎", f"FDV صغير: **{fdv_str}** - 2 , "إمكانية تضاعف عالية", 2)
r.add("⚡", f"ارتفاع 1h: **{change_h1:.1f}%**", 2)
r.add("📈", f"ارتفاع 6h: **{change_h6:.1f}%**", 1)
r.add("❤️", f"ضغط الشراء: **{buy_pressure:.0%}** 2 , "من الصفقات شراء", 2)

if change_h1 >= BREAKOUT_THRESHOLD:
    r.add("🚀", f"***BREAKOUT على قوي {dex.upper()} / {chain.upper()}", 5)

if buys_h1 > 50:
    r.add("👥", f"اهتمام متزايد", 2 , "عدد صفقات الشراء في ساعة: **{buys_h1}** - 2", 2)

if change_h24 > 20:
    r.add("🔥", f"مومنتم استثنائي", 3 , "ارتفاع 24h: **{change_h24:.1f}%** - 3", 3)

# نضع الرابط في extra_data
r.pair_url = pair_url
return True, r

# =====
# 📧 تنسيق الرسالة
# =====

def build_signal_message(coin_name: str, symbol: str, price: float,
                        reason: SignalReason, source: str,
                        extra_url: str = "") -> str:
    now = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC")
    stars = "★" * min(5, max(1, reason.score // 3))

    signal_type = "🚀 BREAKOUT" if reason.score >= 15 else \
        "🔥 HIGH ALPHA" if reason.score >= 10 else \
        "💡 EARLY SIGNAL"

    msg = f"""
    {signal_type}

    🌐 **{coin_name}** ({symbol})
    💰 السعر: `${price:.8f}`
    ★ نقطة {stars} ({reason.score} قوة الإشارة)

```

 الوقت : {now}

 **أسباب الإشارة**
{reason.build()}

 المصدر : {source}
{f" [DexScreener عرض الزوج على {extra_url}]" if extra_url else ""}

 تنبيه: ** هذه إشارات تحليلية وليست نصيحة استثمارية **. (DYOR).
افعل بحثك الخاص دائماً

#AlphaSignals #LowCap #{symbol}
"".strip()
return msg

=====
 مصادر البيانات
=====

```
async def fetch_coingecko_coins(session: aiohttp.ClientSession) -> list:
    """جلب العملات الصغيرة من CoinGecko"""
    url = f"{COINGECKO_BASE}/coins/markets"
    params = {
        "vs_currency": "usd",
        "order": "volume_desc",
        "per_page": 250,
        "page": 1,
        "sparkline": False,
        "price_change_percentage": "1h,7d",
    }
    try:
        async with session.get(url, params=params, timeout=aiohttp.ClientTimeout(
            if r.status == 200:
                return await r.json()
    except Exception as e:
        log.error(f"CoinGecko error: {e}")
    return []
```

```
async def fetch_dexscreener_gainers(session: aiohttp.ClientSession) -> list:
    """جلب أكبر العملات ارتفاعاً من DexScreener"""
    results = []
    chains = ["ethereum", "bsc", "solana", "base", "arbitrum"]
    for chain in chains:
        url = f"{DEXSCREENER_BASE}/search?q=&chainId={chain}"
```

```

try:
    async with session.get(url, timeout=aiohttp.ClientTimeout(total=20))
        if r.status == 200:
            data = await r.json()
            pairs = data.get("pairs") or []
            results.extend(pairs)
except Exception as e:
    log.error(f"DexScreener {chain} error: {e}")
    await asyncio.sleep(0.5) # نجنب rate limit
return results

```

```

# =====
# 🤖 البوت الرئيسي
# =====

```

```

class AlphaSignalsBot:
    def __init__(self):
        self.bot = Bot(token=BOT_TOKEN)
        self.sent_ids: dict[str, float] = {} # id → timestamp آخر إرسال

    def is_on_cooldown(self, coin_id: str) -> bool:
        last = self.sent_ids.get(coin_id)
        if last is None:
            return False
        return (time.time() - last) < (COOLDOWN_HOURS * 3600)

    def mark_sent(self, coin_id: str):
        self.sent_ids[coin_id] = time.time()

    async def send_signal(self, message: str):
        try:
            await self.bot.send_message(
                chat_id=CHANNEL_ID,
                text=message,
                parse_mode=ParseMode.MARKDOWN,
                disable_web_page_preview=False
            )
            log.info(f"✅ إشارة أرسلت بنجاح")
        except Exception as e:
            log.error(f"❌ فشل الإرسال: {e}")

    async def send_startup_message(self):
        msg = f"""
        🤖 **Alpha Signals Bot - التشغيل**

```

✅ البوت يعمل ويراقب السوق الآن

🔍 دقيقة **{SCAN_INTERVAL_MINUTES} يسكان كل

📊 يركز على: Low Cap ($\${\text{MIN_MARKET_CAP_USD}}/1e3:.0f\}$ K - $\${\text{MAX_MARKET_CAP_USD}}/1e6:.0f\}$)

🌐 المصادر: CoinGecko + DexScreener (ETH/BSC/SOL/Base/ARB)

🕒 {datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC")}

⌚ ...انتظر أول إشارة خلال دقائق

```
"".strip()
    await self.send_signal(msg)

async def scan_and_signal(self):
    """دورة مسح واحدة"""
    signals_sent = 0
    async with aiohttp.ClientSession() as session:

        # — 1. CoinGecko —
        log.info("🔍 مسح CoinGecko...")
        cg_coins = await fetch_coingecko_coins(session)
        for coin in cg_coins:
            coin_id = coin.get("id", "")
            if self.is_on_cooldown(coin_id):
                continue
            passed, reason = analyze_coingecko_coin(coin)
            if passed and reason.score >= 8:
                name = coin.get("name", "")
                symbol = coin.get("symbol", "").upper()
                price = coin.get("current_price") or 0
                msg = build_signal_message(
                    name, symbol, price, reason,
                    source="CoinGecko"
                )
                await self.send_signal(msg)
                self.mark_sent(coin_id)
                signals_sent += 1
                await asyncio.sleep(2) # فاصل بين الرسائل

        # — 2. DexScreener —
        log.info("🔍 مسح DexScreener...")
        dex_pairs = await fetch_dexscreener_gainers(session)
        for pair in dex_pairs:
            base = pair.get("baseToken") or {}
            symbol = base.get("symbol", "UNKNOWN").upper()
            addr = base.get("address", symbol)
            uid = f"dex_{addr}"
            if self.is_on_cooldown(uid):
                continue
            passed, reason = analyze_dexscreener_pair(pair)
            if passed and reason.score >= 8:
```



```

        name = base.get("name", symbol)
        price = float(pair.get("priceUsd") or 0)
        url = getattr(reason, "pair_url", "")
        msg = build_signal_message(
            name, symbol, price, reason,
            source=f"DexScreener ({pair.get('chainId','').upper()})",
            extra_url=url
        )
        await self.send_signal(msg)
        self.mark_sent(uid)
        signals_sent += 1
        await asyncio.sleep(2)

log.info(f"✅ إشارة أرسلت - {signals_sent} اكتمل المسح")
if signals_sent == 0:
    log.info("🚀 لا إشارات الآن - السوق هادئ")

async def run(self):
    log.info("🚀 تشغيل Alpha Signals Bot...")
    await self.send_startup_message()
    while True:
        try:
            await self.scan_and_signal()
        except Exception as e:
            log.error(f"خطأ في دورة المسح: {e}")
            log.info(f"⌚ {SCAN_INTERVAL_MINUTES} دقيقة...انتظار")
            await asyncio.sleep(SCAN_INTERVAL_MINUTES * 60)

# =====
# 🎬 نقطة البداية
# =====

if __name__ == "__main__":
    if BOT_TOKEN == "YOUR_BOT_TOKEN_HERE":
        print("❌ أولاً BOT_TOKEN ضع توكن البوت في متغير")
        exit(1)
    if CHANNEL_ID == "@YourChannelUsername":
        print("❌ أولاً CHANNEL_ID ضع اسم القناة في متغير")
        exit(1)

    bot = AlphaSignalsBot()
    asyncio.run(bot.run())

```